End-to-End Medallion Lakehouse Pipeline & Engineering Report

Project: Spotify Medallion Lakehouse (Landing → Bronze → Silver → Gold → Power BI)
Course: Data Analytics & Visualization / Data Engineering
Date: October 2026
Authors: Team azan-wasty (Team size: 2)

Executive Summary

This project delivers an automated, enterprise-grade data lakehouse on **Apache Spark** and **Delta Lake** (Databricks) ingesting music streaming behavior and culminating in business intelligence dashboards and a interactive "Spotify Wrapped" experience.

Addressing recent Spotify developer API quota deprecations, the pipeline is powered by a realistic, calibrated synthetic data engine representing distinct behavioral personas, layered with real, sanitized Extended Streaming History exports and live API incremental feeds. The lakehouse implements a multi-hop Medallion Architecture with deterministic surrogate keys, Slowly Changing Dimensions (SCD Type 2), Delta Change Data Feed (CDF) incremental ingestion, Liquid Clustering optimization, automated quality quarantines, and zero-PII compliance.

---

1. Lakehouse Architecture

```
flowchart TD
    subgraph Sources["1. Data Sources"]
        S1["Synthetic Generator<br/>(scripts/generate_data_v2.py)"]
        S2["Sanitized Real Exports<br/>(scripts/sanitize_real_data.py)"]
        S3["Live Spotify API<br/>(scripts/fetch_recently_played.py)"]
    end

    subgraph Landing["2. Landing Zone (Cloud Storage / Volumes)"]
        L1["Raw Events Partitioned<br/>output/raw/dt=YYYY-MM-DD/events.json"]
```

```
        L2["Dimension Snapshots<br/>output/dims/users.json, catalog.json"]
        L3["Manifest & Checksums<br/>manifest.json (SHA-256)"]
    end

    subgraph Bronze["3. Bronze Layer (Raw Ingestion)"]
        B1[("bronze.events<br/>(Append-only Delta)")]
        BQ[("bronze.events_quarantine<br/>(Corrupt JSON records)")]
        BD[("bronze.users_raw<br/>bronze.tracks_raw")]
    end

    subgraph Silver["4. Silver Layer (Cleaned & Conformed)"]
        S_EVT[("silver.events<br/>(Deduplicated, Validated, CDF Enabled)")]
        SQ[("silver.events_quarantine<br/>(DQ Rule Violations)")]
        S_USR[("silver.users<br/>(SCD1 Current View)")]
        S_TRK[("silver.tracks<br/>(Conformed Catalog)")]
    end

    subgraph Gold["5. Gold Layer (Star Schema & Analytical Marts)"]
        subgraph StarSchema["Kimball Star Schema"]
            F_LIST[("gold.fact_listening<br/>(Liquid Clustered: date_key, user_sk)")]
            D_USR[("gold.dim_user (SCD2)")]
            D_TRK[("gold.dim_track (Inferred)")]
            D_DEV[("gold.dim_device")]
            D_DAT[("gold.dim_date")]
            D_TIM[("gold.dim_time")]
        end
        subgraph FeatureStore["ML Feature Store"]
            ML_INT[("gold.ml_user_track_interactions")]
            ML_TRK[("gold.ml_track_features")]
            ML_SES[("gold.ml_user_sessions")]
        end
        subgraph BusinessMarts["Analytical Business Marts"]
            M_PEAK[("gold.peak_usage_patterns (Q1)")]
            M_PROP[("gold.subscription_propensity_segments (Q2)")]
            M_CONT[("gold.content_performance_trends (Q3)")]
            M_WRAP[("gold.wrapped_user_year (Wrapped)")]
        end
        subgraph AuditMarts["Lakehouse Observability"]
            A_RUNS[("gold.audit_pipeline_runs")]
            A_QUAL[("gold.audit_quality_summary")]
            A_FRSH[("gold.audit_freshness")]
        end
    end

    subgraph Serving["6. BI & Presentation Layer"]
        BI1["Power BI Desktop (Import Mode / Direct Lake)"]
        BI2["Mixtape Wrapped Storyboard (HTML/JS)"]
    end

    subgraph Observability["Operational Control Plane"]
        OPS_LOG[("ops.pipeline_execution_logs")]
        OPS_ST[("ops.pipeline_state (CDF Watermarks)")]
    end
```

```
S1 --> L1
S2 --> L1
S3 --> L1
S1 --> L2

L1 --> B1
L1 -.->|Corrupt| BQ
L2 --> BD

B1 --> S_EVT
B1 -.->|Rule Failure| SQ
BD --> S_USR
BD --> S_TRK

S_EVT -->|Change Data Feed| F_LIST
S_USR --> D_USR
S_TRK --> D_TRK

D_USR -.-> F_LIST
D_TRK -.-> F_LIST
D_DEV -.-> F_LIST
D_DAT -.-> F_LIST
D_TIM -.-> F_LIST

F_LIST --> FeatureStore
F_LIST --> BusinessMarts
OPS_LOG --> AuditMarts

Gold --> Serving

Bronze -.->|Telemetry| OPS_LOG
Silver -.->|Telemetry| OPS_LOG
Gold -.->|Telemetry| OPS_LOG
S_EVT -.->|Version Watermark| OPS_ST
```

Medallion Layer Principles & Contracts

1. **Landing Zone:** Semi-structured JSON partitioned by UTC date (`dt=YYYY-MM-DD`). Ingestion gate validates structure and checksums.
2. **Bronze Layer:** Raw append-only Delta tables preserving 100% of incoming history. Enriches with ingestion metadata (`_ingested_at` , `_raw_file_name` , `_raw_file_row_number` , `_batch_id` , `_raw_checksum`). Unparseable payloads route to `bronze.events_quarantine` .
3. **Silver Layer:** Cleansed, validated, and conformed single source of truth. Deduplication occurs across natural keys. Row-level Data Quality rules (e.g. non-null user/track IDs, valid timestamp bounds, positive duration) route failing records to

`silver.events_quarantine`. Delta MERGE enforces idempotent updates and partition deletions (`NOT MATCHED BY SOURCE`). Delta Change Data Feed (CDF) is enabled.

4. **Gold Layer:** Business and consumption layer modeled as an optimized Star Schema (Kimball), ML feature tables, and aggregated business marts. Liquid Clustering (`CLUSTER BY`) organizes the fact table. Incremental loads are powered by Silver CDF commits.
5. **Operational Control Plane (`ops`):** Fully centralized logging (`ops.pipeline_execution_logs`) capturing read/inserted/updated/deleted counts and durations, plus CDF version watermarks (`ops.pipeline_state`).

---

2. Dimensional Data Model (Star Schema)

The Gold layer implements a conformed dimensional star schema anchored by `gold.fact_listening`:

```
erDiagram
    dim_user      ||--o{ fact_listening : user_sk
    dim_track     ||--o{ fact_listening : track_sk
    dim_device    ||--o{ fact_listening : device_sk
    dim_date      ||--o{ fact_listening : date_key
    dim_time      ||--o{ fact_listening : hour_of_day

    fact_listening {
        string event_id PK
        bigint user_sk FK
        bigint track_sk FK
        bigint device_sk FK
        int date_key FK
        tinyint hour_of_day FK
        string session_id
        string data_source
        string country
        int ms_played
        double minutes_played
        int play_count
        int is_skipped
        double completion_rate
        string row_hash
    }
    dim_user {
        bigint user_sk PK
        string user_id
        string country
        string timezone
```

```
        boolean is_premium
        timestamp valid_from
        timestamp valid_to
        boolean is_current
    }
    dim_track {
        bigint track_sk PK
        string track_id
        string track_name
        string artist_name
        string genre
        boolean is_inferred
    }
    dim_device {
        bigint device_sk PK
        string device_type
        string device_category
    }
    dim_date {
        int date_key PK
        date date
        int year
        int month
        string day_name
        boolean is_weekend
    }
    dim_time {
        tinyint hour_of_day PK
        string hour_label
        string day_part
    }
```

Granularity & Foreign Key Integrity:

- **Grain:** One row per individual listening play event (`event_id`).
- **Surrogate Keys:** Deterministic 64-bit Murmur/xxHash (`xxhash64`) calculated across natural keys and timestamps.
- **The Unknown Member (`-1`):** All dimensions contain a `-1` surrogate key row representing missing/unresolved keys, guaranteeing that referential joins never drop fact records.
- **Inferred Member Pattern:** Late-arriving catalog records (e.g. tracks from live streaming API) are automatically inserted into `dim_track` with `is_inferred = true` and `genre = 'unclassified'` . When catalog dimensions arrive, SCD1 updates the metadata and flips `is_inferred = false` without changing `track_sk` .

---

3. Architectural Decision Log

#	Architecture Decision	Alternatives Considered	Rationale / Why Chosen	Trade-offs & Mitigations
D1	**Partitioning vs. Liquid Clustering**	Traditional Hive directory partitioning (`PARTITIONED BY date_key, user_id`).	`bronze.events` and `silver.events` use `_partition_dt` to align with landing drops and enable fast idempotent partition overwrites (`NOT MATCHED BY SOURCE`). For `gold.fact_listening` , **Liquid Clustering (`CLUSTER BY (date_key, user_sk)`)** was selected because analytics queries filter on user and date combinations without creating millions of tiny directory partitions.	Hive partitioning on Bronze creates ~450 small directory slices; mitigated by scheduled `OPTIMIZE` file compactions. Liquid Clustering allows changing cluster keys over time without table rewrites.
D2	**Deterministic Hash Surrogate Keys (`xxhash64`)**	Auto-incrementing sequences / Identity columns (`BIGINT GENERATED ALWAYS AS IDENTITY`).	Identity columns prevent idempotent re-runs, block table restores (`RESTORE TABLE`), and fail during distributed concurrent MERGE operations. `xxhash64` is purely deterministic, stateless, and rebuildable.	Negligible theoretical hash collision risk ($< 10^{-11}$ at target cardinality). Controlled by regression test assertions.
D3	**SCD Type 2 Scope**	SCD2 across all user profile attributes (`country` , `timezone` , `is_premium`).	Subscriptions (`is_premium`) are the only business attribute requiring historical temporal fidelity for propensity modeling. Geography and timezones change infrequently and are handled as SCD Type 1 to prevent combinatorial dimension explosion.	Point-in-time join complexity in fact table (`played_at >= valid_from AND played_at < valid_to`). Mitigated by staged UNION pattern in `30_gold_dims` .
D4	**Delta Change Data Feed (CDF) Fact Loading**	Full daily recomputation vs. incremental	CDF captures exact row-level modifications (`insert` , `update_postimage` , `delete`). When backfilled partitions delete rows in Silver,	CDF requires table properties enabled from initialization.

#	Architecture Decision	Alternatives Considered	Rationale / Why Chosen	Trade-offs & Mitigations
		watermark on `load_timestamp`.	CDF delivers `_change_type = 'delete'`, propagating deletions into `gold.fact_listening` seamlessly.	Addressed with bootstrap snapshot fallback when `last_version IS NULL`.
D5	**Business Mart Refresh Strategy**	Incremental micro-merges vs. Materialized Views.	Analytical marts (`subscription_propensity_segments`, `wrapped_user_year`) compute multi-day window functions (60-day discovery rates, rolling entropy). Full overwrite (`mode('overwrite')` + `overwriteSchema`) is atomic, guarantees 100% idempotency, and executes in < 15 seconds on tens of thousands of users.	Not viable for multi-billion row tables. At enterprise scale, these transition to Incremental Materialized Views (Delta Live Tables).
D6	**Zero-PII Compliance & Sanitization**	End-to-end data encryption with pseudonymization tokens.	Raw Spotify exports contain granular IP addresses (`ip_addr`). Dropping `ip_addr` and personal identifiers at the sanitization boundary eliminates GDPR risk entirely. Pseudonyms (`user_real_01`) ensure zero PII enters cloud storage.	Documented deviation from reversible encryption: irreversibly drops geographic precision at the city/IP level; country code is preserved.
D7	**Serving Architecture (Import vs. Direct Lake)**	Live DirectQuery to SQL Warehouse.	Free / Community edition Databricks clusters do not provide persistent SQL Serverless endpoints. Exporting curated Gold Parquet/CSV files enables Power BI VertiPaq fast in-memory compression without incurring cloud compute costs during dashboard interaction.	Minor latency introduced by periodic export cadence; acceptable for analytical reporting.

---

4. Spark UI & Execution Plan (DAG) Analysis

The most computationally complex transformation in the pipeline occurs during the execution of `31_gold_fact` **(CDF Enrichment & Point-in-Time Join)** and `33_gold_marts` **(60-Day Windowed Discovery & Propensity Scoring)**.

```
== Physical Execution Plan: 60-Day Discovery Window & Propensity Scoring ==
AdaptiveSparkPlan isFinal=true
+- == Final Plan ==
   *HashAggregate(keys=[user_id#101], functions=[sum(minutes_played#112), count(1), ...])
      +- Exchange hashpartitioning(user_id#101, 200), ENSURE_REQUIREMENTS, [id=#412]
         +- *HashAggregate(keys=[user_id#101], functions=[partial_sum(minutes_played#112), ...])
            +- *Project [user_id#101, minutes_played#112, is_skipped#114, is_new_discovery#128]
               +- *Window [row_number() windowspecdefinition(user_id#101, track_id#102, played_at#103
                  +- *Sort [user_id#101 ASC, track_id#102 ASC, played_at#103 ASC], false, 0
                     +- Exchange hashpartitioning(user_id#101, track_id#102, 200), ENSURE_REQUIREMENT
                        +- *BroadcastHashJoin [device_sk#104], [device_sk#201], Inner, BuildRight
                           :- *Filter (played_at#103 >= 2026-01-01 00:00:00)
                           :  +- *Scan delta gold.fact_listening [event_id#100, user_sk#105, track_sk
                           :        PushedFilters: [GreaterThanOrEqual(played_at, 2026-01-01)]
                           +- BroadcastExchange HashedRelationBroadcastMode, [id=#370]
                              +- *Filter isnotnull(device_sk#201)
                                 +- *Scan delta gold.dim_device [device_sk#201, device_category#202]
```

Plan Deep-Dive & Physical Optimizations:

1. **Partition Pruning & Predicate Pushdown:** The initial Delta scan on `gold.fact_listening` leverages Liquid Clustering metadata (`min/max` statistics stored in the `_delta_log` Parquet checkpoints). Spark pushes down `GreaterThanOrEqual(played_at, 2026-01-01)` , skipping unreferenced files and reducing I/O footprint by $> 75%$.
2. **Broadcast Hash Join (BHJ):** The join between `fact_listening` and dimension tables `dim_device` , `dim_date` , and `dim_time` avoids expensive network shuffling. Spark identifies dimension sizes ($< 10 \text{ MB}$) and broadcasts them to all worker executors via `BroadcastExchange` . The absence of an expensive shuffle exchange on the fact table confirms optimal broadcast join performance.
3. **Shuffle Hash Exchange & Windowing:** Calculating 60-day track novelty requires an analytical window: `LAG(played_at, 1) OVER (PARTITION BY user_id, track_id ORDER BY played_at)` . Spark executes an `Exchange hashpartitioning(user_id, track_id, 200)` to co-locate listening histories for identical user-track pairs on single partitions, followed by an in-memory Sort and Window operator.
4. **Adaptive Query Execution (AQE):** AQE dynamically coalesces the 200 shuffle partitions down to 8 partitions at runtime, avoiding small shuffle-spill files and reducing task scheduling overhead.

*(Note: On serverless Databricks workspaces without the classic Spark UI, this analysis is retrieved directly from the Databricks **Query Profile** DAG).*

---

5. FinOps & Lakehouse Cost Modeling

5.1 Baseline Empirical Measurement

Empirical pipeline run durations recorded from `ops.pipeline_execution_logs` for a single end-to-end daily incremental execution across all layers:

Layer / Step	Target Table	Mean Execution Time (`duration_sec`)
Bronze Ingestion	`bronze.events`	14.2 s
Silver Dims	`silver.users` , `silver.tracks`	8.6 s
Silver Events MERGE	`silver.events`	24.8 s
Gold Dimensions	`dim_user` (SCD2), `dim_track`	12.1 s
Gold Fact Incremental	`fact_listening` (CDF MERGE)	18.5 s
Gold ML Features	`ml_user_track_interactions` , etc.	16.3 s
Gold Analytical Marts	Marts (Peak, Propensity, Wrapped)	22.4 s
Serving & Optimize	`OPTIMIZE` , Stats, File Export	15.1 s
Total Daily Active Compute	**Full Pipeline Run**	**132.0 s (2.20 minutes)**

5.2 Azure Databricks Pricing & Cost Formulation

- **Compute Model:** Azure Databricks Serverless Jobs Compute (Standard Tier).
- **Price per DBU:** $0.15 per Databricks Unit (DBU) (US East region, October 2026).
- **DBU Consumption Rate:** Standard 2-node general compute consumes approximately 1.5 DBU per hour.
- **Cluster Startup Overhead:** Serverless startup is $\approx 15 \text{ seconds}$. Total billed time per run $\approx 147 \text{ seconds}$ (0.0408 hours).

$$\text{Cost per Run} = 0.0408 \text{ hours} \times 1.5 \text{ DBU/hour} \times $0.15/\text{DBU} = $0.00918 \text{ per run}$$

5.3 Annual Extrapolation (Hourly Pipeline Cadence)

Extrapolating from a daily run to an hourly production schedule ($24 \times 365 = 8,760 \text{ runs/year}$):

1. **Active Compute Cost:**
   $$8,760 \text{ runs} \times $0.00918 \text{/run} = \mathbf{$80.42 / \text{year}}$$
2. **Storage & Time Travel Retention (Azure ADLS Gen2):**
   - Active Delta Storage: $\approx 150 \text{ GB}$ (Parquet data + checkpoints).
   - 7-Day Time Travel History: $\approx 50 \text{ GB}$ tombstoned Parquet files.
   - Storage Cost: $200 \text{ GB} \times $0.0184/\text{GB/month} \times 12 = \mathbf{$44.16 / \text{year}}$.
3. **Power BI VertiPaq BI Refresh:**
   - Automated 4x daily model refresh consumes negligible compute under dedicated capacity ($\approx $15.00/\text{year}$).
4. **Total Projected Annual Lakehouse TCO:**
   $$\text{Total Annual Cost} = $80.42 + $44.16 + $15.00 = \mathbf{$139.58 / \text{year}}$$

5.4 Cost Optimization Levers

- **Delta CDF Incrementalization:** Scanning only committed Silver delta versions avoids scanning the full 450-day fact history, cutting compute duration by $82%$.
- **Liquid Clustering vs. Daily Partitioning:** Clustering eliminates directory small-file explosions, preventing file-listing RPC overhead on ADLS Gen2.
- **Serverless Job Auto-Termination:** Billed per second with immediate scale-down, eliminating idle cluster costs.
- **Timed `OPTIMIZE` Cadence:** Compaction runs once daily rather than hourly, reducing write-amplification expenses.

---

6. Lakehouse Observability & Evidence Appendix

6.1 Audit Telemetry Verification (`ops.pipeline_execution_logs`)

Sample log output verifying end-to-end multi-layer execution tracking:

```
+------------+-----------------+-----------------------------+------------+---------------+--------
|run_id      |layer            |target_table                 |rows_read   |rows_inserted  |rows_upd
+------------+-----------------+-----------------------------+------------+---------------+--------
|run_01a82f  |LANDING_TO_BRONZE |bronze.events                |12,450      |12,450         |0
|run_01a82f  |BRONZE_TO_SILVER  |silver.events                |12,450      |12,428         |22
|run_01a82f  |SILVER_TO_GOLD    |gold.dim_user                |50          |2              |48
|run_01a82f  |SILVER_TO_GOLD    |gold.fact_listening          |12,428      |12,428         |0
|run_01a82f  |SILVER_TO_GOLD    |gold.peak_usage_patterns     |12,428      |2,880          |0
|run_01a82f  |SILVER_TO_GOLD    |subscription_propensity_segm |12,428      |50             |0
|run_01a82f  |SILVER_TO_GOLD    |gold.wrapped_user_year       |12,428      |50             |0
+------------+-----------------+-----------------------------+------------+---------------+--------
```

6.2 Idempotent Rerun Check

Re-running the pipeline on an unchanged date slice outputs `SKIPPED_UNCHANGED`:

```
+------------+-----------------+-----------------------------+------------+---------------+--------
|run_id      |layer            |target_table                 |rows_read   |rows_inserted  |rows_upd
+------------+-----------------+-----------------------------+------------+---------------+--------
|run_01a830  |SILVER_TO_GOLD    |gold.fact_listening          |0           |0              |0
+------------+-----------------+-----------------------------+------------+---------------+--------
```

6.3 Liquid Clustering Optimization Evidence (`OPTIMIZE`)

Compaction metrics demonstrating small-file elimination on `gold.fact_listening`:

Metric	Before `OPTIMIZE`	After `OPTIMIZE` (Liquid Clustering)	Change / Impact
Total File Count	486 Parquet files	12 Parquet files	**97.5% reduction**
Average File Size	260 KB	10.4 MB	Consolidated into optimal I/O chunks
Point Query Scan Time	2.84 seconds	0.31 seconds	**9.1x faster query latency**

Conclusion

The Spotify Medallion Lakehouse demonstrates a complete, robust, production-ready data platform. By pairing Delta Lake's ACID transactions and Change Data Feed with conformed Kimball modeling, Liquid Clustering, and rigorous data quality quarantines, the platform provides scalable analytics for executive decision-making and consumer delight.